Write a static method insertEveryN that takes a string str, a character ch, and an integer n. The method should return a new string where the character ch is inserted after every n characters in str.

Given Test cases:

insertEveryN("helloworld", '-', 3) → "hel-low-orl-d"

insertEveryN("javaisfun", '*', 2) → "ja*va*is*fu*n"

For example:

Test	Result
`System.out.println(insertEveryN("helloworld", '-', 3));`	`hel-low-orl-d`
`System.out.println(insertEveryN("javaisfun", '*', 2));`	`ja*va*is*fu*n`

Answer: (penalty regime: 0 %)

Reset answer

```java
public static String insertEveryN(String str, char ch, int n) {
    String result = "";

    for (int i = 0; i < str.length(); i++) {
        result += str.charAt(i);

        if ((i + 1) % n == 0 && i != str.length() - 1) {
            result += ch;
        }
    }

    return result;
}

```

Check

A warehouse manager needs to conduct an advanced inventory check. The warehouse has 4 aisles, each containing 6 shelves. Some shelves may be partially filled, overfilled, or empty. The following conditions need to be met:

- Shelves with odd numbers are always empty.
- Even-numbered shelves can be either partially filled or overfilled:
  - If the even-numbered shelf is divisible by 4, it is overfilled.
  - If the even-numbered shelf is not divisible by 4, it is partially filled.

Complete the code to print the status of each shelf in the warehouse according to these conditions.

```
public class WarehouseInventory {
  public static void main(String[] args) {
    for (int aisle = 1; aisle <= 4; aisle++) {
      for (int shelf = 1;__(1)___; shelf++) {
        if (__(2)__ != 0) {
          System.out.println("Aisle " + aisle + ", Shelf " + shelf + ": Empty");
        } else if (__(3)__ == 0) {
          System.out.println("Aisle " + aisle + ", Shelf " + shelf + ": Overfilled");
        } __(4)__ {
          System.out.println("Aisle " + aisle + ", Shelf " + shelf + ": Partially filled");
        }
      }
    }
  }
}
```

(1) shelf <= 6

(2) shelf % 2

(3) shelf % 4

(4) else

Searching for prime numbers: we want to find all prime numbers that are less than 100. In the following implementation, we use array A[100] to searching for these prime numbers. Please fill in the missing lines.

The search is performed by order. When a prime number p is detected, the algorithm will label all numbers q as non-prime, where q = k*p, k>1. For example, if p=3, then all numbers in {6, 9, 12,..., 96, 99} will be labeled as non-prime numbers. Finally, when the algorithm ends, for every prime number i, we have A[i]=i; otherwise we have A[i]=0.

```
public static void main(String[] args) {
        // Initialization
        int[] A = new int[100];
        for (int i = 0; i < 100; i++) {
                ____(1)____;
        }
        A[0] = 0; // 0 is not a prime number
        A[1] = 0; // 1 is not a prime number

        // Detecting prime numbers
        // If i is a prime number, then A[i]=i.
        // Otherwise, A[i]=0.
        for (int i = 2; i < 100; i++) {
                if (A[i] != 0) {
                        for (int j = 2 * i; j < 100; ____(2)____) {
                                ____(3)____;
                        }
                }

                // Output all prime numbers
                for (i = 0; i < 100; i++) {
                        // add your code here
                        if (____(4)____) {
                                System.out.println(i + " ");
                        }
                }
        }
}
```

(1) A[i] = i

(2) j+=i

(3) A[j] = 0

(4) A[i] != 0

Searching for prime numbers: we want to find all prime numbers that are less than 100. In the following implementation, we use array A[100] to searching for these prime numbers. Please fill in the missing lines.

The search is performed by order. When a prime number p is detected, the algorithm will label all numbers q as non-prime, where q = k*p, k>1. For example, if p=3, then all numbers in {6, 9, 12,..., 96, 99} will be labeled as non-prime numbers. Finally, when the algorithm ends, for every prime number i, we have A[i]=i; otherwise we have A[i]=0.

```
public static void main(String[] args) {
        // Initialization
        int[] A = new int[100];
        for (int i = 0; i < 100; i++) {
                ____(1)____;
        }
        A[0] = 0; // 0 is not a prime number
        A[1] = 0; // 1 is not a prime number

        // Detecting prime numbers
        // If i is a prime number, then A[i]=i.
        // Otherwise, A[i]=0.
        for (int i = 2; i < 100; i++) {
                if (A[i] != 0) {
                        for (int j = 2 * i; j < 100; ____(2)____) {
                                ____(3)____;
                        }
                }

                // Output all prime numbers
                for (i = 0; i < 100; i++) {
                        // add your code here
                        if (____(4)____) {
                                System.out.println(i + " ");
                        }
                }
        }
}
```

(1) A[i] = i

(2) j+=i

(3) A[j] = 0

(4) A[i] != 0

Write a method removeDoubledLetters that takes a string as its arguments and returns a new string with all doubled letters in the string replaced by a single letter. For example, if you call

removeDoubledLetters("zookeeper")

your method should return the string "zokeper". Similarly, if you call

removeDoubledLetters("footpath")

your method should return "fotpath".

In writing your solution, you should keep in mind the following:

- You do not need to write a complete program. All you need is the definition of the method removeDoubledLetters that returns the desired result.
- You may assume that all letters in the string are lower case so that you don't have to worry about changes in capitalization.
- You may assume that no letter appears more than twice in a row.

Given Test cases:
removeDoubledLetters("footpath") -> "fotpath"
removeDoubledLetters("zookeeper") -> "zokeper"

For example:

Test	Result
`System.out.println(removeDoubledLetters("footpath"));`	fotpath
`System.out.println(removeDoubledLetters("zookeeper"));`	zokeper

Answer: (penalty regime: 0 %)

Reset answer

```java
public static String removeDoubledLetters(String str) {
  if (null == str || str.length() == 0) return "";
  if (str.length() == 1) return str;

  String s = "";
  for (int i = 0; i < str.length(); i++) {
    if (i + 1 > str.length() - 1) s += str.charAt(i);
    else if (str.charAt(i) == str.charAt(i + 1)) continue;
    else s += str.charAt(i);
  }
  return s;
}
```

Check

A warehouse manager needs to conduct an advanced inventory check. The warehouse has 4 aisles, each containing 6 shelves. Some shelves may be partially filled, overfilled, or empty. The following conditions need to be met:

- Shelves with odd numbers are always empty.
- Even-numbered shelves can be either partially filled or overfilled:
  - If the even-numbered shelf is divisible by 4, it is overfilled.
  - If the even-numbered shelf is not divisible by 4, it is partially filled.

Complete the code to print the status of each shelf in the warehouse according to these conditions.

```
public class WarehouseInventory {
  public static void main(String[] args) {
    for (int aisle = 1; aisle <= 4; aisle++) {
      for (int shelf = 1;__(1)___; shelf++) {
        if (__(2)__ != 0) {
          System.out.println("Aisle " + aisle + ", Shelf " + shelf + ": Empty");
        } else if (__(3)__ == 0) {
          System.out.println("Aisle " + aisle + ", Shelf " + shelf + ": Overfilled");
        } __(4)__ {
          System.out.println("Aisle " + aisle + ", Shelf " + shelf + ": Partially filled");
        }
      }
    }
  }
}
```

(1) shelf <= 6

(2) shelf % 2

(3) shelf % 4

(4) else

Consider the following code snippet. What will happen when the code is executed?

```
public class Main {
  public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4, 5};

    for (int i = 0; i <= numbers.length; i++) {
      System.out.println(numbers[i]);
    }
  }
}
```

Select one:

- ○ a. The code will result in a compilation error
- ◉ b. The code will throw an ArrayIndexOutOfBoundsException
- ○ c. The code will print all the numbers except the last one
- ○ d. The code will print all the numbers in the array

Clear my choice

You are managing a list of products for a grocery store. Each product is represented by a non-empty string, and you need to categorize them by their first letter. Your task is to write a method called categorizeProductsByFirstLetter that processes the product list and returns a Map<Character, String> where:

- Key: The first letter of the product, the type is Character (char)
- Value: All the product names starting with that letter, concatenated together and separated by a semicolon (;), in the order they appear in the list. The type is String

The following are imported: Map, HashMap, Arrays, List, ArrayList.
Do ***not*** import other libraries.

Given Test cases:

System.out.println(categorizeProductsByFirstLetter (new ArrayList<>(Arrays.asList("Apple", "Avocado", "Banana", "Broccoli", "Cherry", "Carrot")))) → {A=Apple;Avocado, B=Banana;Broccoli, C=Cherry;Carrot}

System.out.println(categorizeProductsByFirstLetter (new ArrayList<>(Arrays.asList("Milk", "mango", "Cheese", "Butter", "banana")))) → {B=Butter;banana, C=Cheese, M=Milk;mango}

For example:

Test	R
System.out.println(categorizeProductsByFirstLetter (new ArrayList<>(Arrays.asList("Apple", "Avocado", "Banana", "Broccoli", "Cherry", "Carrot"))))	{
System.out.println(categorizeProductsByFirstLetter (new ArrayList<>(Arrays.asList("Milk", "mango", "Cheese", "Butter", "banana"))))	{

Answer: (penalty regime: 0 %)

Reset answer

```java
public static Map<Character, String> categorizeProductsByFirstLetter(List<String> products) {
    Map<Character, String> productMap = new HashMap<>();
    for (String product : products) {
        char firstLetter = product.toUpperCase().charAt(0);
        if (productMap.containsKey(firstLetter)) {
            String existingProducts = productMap.get(firstLetter);
            productMap.put(firstLetter, existingProducts + ";" + product);
        } else {
            productMap.put(firstLetter, product);
        }
    }
    return productMap;
}

```

Check

What is the output of the following program?

```
class Base {
  public void Print() {
    System.out.println("Base");
  }
}

class Derived extends Base {
  public void Print() {
    System.out.println("Derived");
  }
}

public class Main {
  public static void DoPrint(Base obj) {
    obj.Print();
  }

  public static void main(String[] args) {
    Base a = new Base();
    Base b = new Derived();
    Derived c = new Derived();
    DoPrint(a);
    DoPrint(b);
    DoPrint(c);
  }
}
```

Select one:

- (selected) a. Base
  Derived
  Derived
- b. Compilation Error
- c. Derived
  Derived
  Derived
- d. Base
  Derived
  Base

Clear my choice

Write a static method sumGreaterThan that takes an array of integers and a number n. The method should return the sum of all elements in the array that are greater than n. You must use recursion and must **not** use loops to solve this problem.
Hint: Use recursion with a helper method.

Given Test cases:

sumGreaterThan(new int[]{1, 5, 3, 9, 2}, 4) → 14 // (5 + 9)

sumGreaterThan(new int[]{7, 2, 8, 6, 2}, 5) → 21 // (7 + 8 + 6)

For example:

Test	Result
`System.out.println(sumGreaterThan(new int[]{1, 5, 3, 9, 2}, 4));`	14
`System.out.println(sumGreaterThan(new int[]{7, 2, 8, 6, 2}, 5));`	21

Answer: (penalty regime: 0 %)

Reset answer

```java
public static int sumGreaterThan(int[] arr, int n) {
    return sumGreaterThanHelper(arr, n, 0);
}

private static int sumGreaterThanHelper(int[] arr, int n, int index) {
    if (index == arr.length) {
        return 0;
    }

    int current = (arr[index] > n) ? arr[index] : 0;
    return current + sumGreaterThanHelper(arr, n, index + 1);
}

```

Check

Write a static method sumGreaterEqual that takes an array of integers and a number n. The method should return the sum of all elements in the array that are greater than or equal to n.
You must use recursion and must **not** use loops to solve this problem.
Hint: Use recursion with a helper method.

Given Test cases:

sumGreaterEqual(new int[]{1, 5, 3, 9, 2}, 4) → 14 // (5 + 9)

sumGreaterEqual(new int[]{7, 2, 8, 6, 2}, 5) → 21 // (7 + 8 + 6)

For example:

Test	Result
`System.out.println(sumGreaterEqual(new int[]{1, 5, 3, 9, 2}, 4));`	14
`System.out.println(sumGreaterEqual(new int[]{7, 2, 8, 6, 2}, 5));`	21

Answer: (penalty regime: 0 %)

Reset answer

```java
public static int sumGreaterEqual(int[] arr, int n) {
    return sumGreaterEqualHelper(arr, n, 0);
}

private static int sumGreaterEqualHelper(int[] arr, int n, int index) {
    if (index == arr.length) {
        return 0;
    }

    int current = (arr[index] > n) ? arr[index] : 0;
    return current + sumGreaterEqualHelper(arr, n, index + 1);
}

```

Check

Design a class named QuadraticEquation for a quadratic equation $ax^2 + bx + c = 0$.
The class contains:

a) Private data fields a, b, and c that represents three coefficients, which can have decimal values.
b) A constructor for the arguments a, b, and c.
c) Three get methods for a, b, and c.
d) A method named getDiscriminant() that returns the discriminant, which is $b^2 - 4ac$.
e) The methods named getRoot1() and getRoot2() for returning two roots of the equation

$$r_1 = \frac{-b+\sqrt{b^2-4ac}}{2a} \text{ and } r_2 = \frac{-b-\sqrt{b^2-4ac}}{2a}$$

These methods are useful only if the discriminant is nonnegative, or a RuntimeException should be thrown with a message "Discriminant is negative".

Given Test cases:
(new QuadraticEquation(1.0, 1.0, -6.0)).getDiscriminant() -> 25.0
(new QuadraticEquation(1.0, 1.0, -6.0)).getRoot1() -> 2.0
(new QuadraticEquation(1.0, 1.0, -6.0)).getRoot2() -> -3.0

For example:

Test	Result
`System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getDiscriminant());`	25.0
`System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getRoot1());`	2.0
`System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getRoot2());`	-3.0

Answer: (penalty regime: 0 %)

Reset answer

```java
public class QuadraticEquation {

    private double a;
    private double b;
    private double c;

    public QuadraticEquation(double a, double b, double c) {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    public double getA() {
        return a;
    }

    public double getB() {
        return b;
    }

    public double getC() {
        return c;
    }

    public double getDiscriminant() {
        return b * b - 4 * a * c;
    }

    public double getRoot1() {
        double discriminant = getDiscriminant();
        if (discriminant < 0) throw new RuntimeException("Discriminant is negative");

        double res = (-b + Math.sqrt(discriminant)) / 2 / a;
        return res;
    }

    public double getRoot2() {
        double discriminant = getDiscriminant();
        if (discriminant < 0) throw new RuntimeException("Discriminant is negative");

        double res = (-b - Math.sqrt(discriminant)) / 2 / a;
        return res;
    }
}
```

Check

Design a class named QuadraticEquation for a quadratic equation $ax^2 + bx + c = 0$.
The class contains:

a) Private data fields a, b, and c that represents three coefficients, which can have decimal values.
b) A constructor for the arguments a, b, and c.
c) Three get methods for a, b, and c.
d) A method named getDiscriminant() that returns the discriminant, which is $b^2 - 4ac$.
e) The methods named getRoot1() and getRoot2() for returning two roots of the equation

$$r_1 = \frac{-b+\sqrt{b^2-4ac}}{2a} \text{ and } r_2 = \frac{-b-\sqrt{b^2-4ac}}{2a}$$

These methods are useful only if the discriminant is nonnegative, or a IllegalArgumentException should be thrown with
a message "Discriminant must not be negative".

Given Test cases:
(new QuadraticEquation(1.0, 1.0, -6.0)).getDiscriminant() -> 25.0
(new QuadraticEquation(1.0, 1.0, -6.0)).getRoot1() -> 2.0
(new QuadraticEquation(1.0, 1.0, -6.0)).getRoot2() -> -3.0

For example:

Test	Result
`System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getDiscriminant());`	25.0
`System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getRoot1());`	2.0
`System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getRoot2());`	-3.0

Answer: (penalty regime: 0 %)

Reset answer

```java
public class QuadraticEquation {

    private double a;
    private double b;
    private double c;

    public QuadraticEquation(double a, double b, double c) {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    public double getA() {
        return a;
    }

    public double getB() {
        return b;
    }

    public double getC() {
        return c;
    }

    public double getDiscriminant() {
        return b * b - 4 * a * c;
    }

    public double getRoot1() {
        double discriminant = getDiscriminant();
        if (discriminant < 0) throw new IllegalArgumentException("Discriminant must not be negative");

        double res = (-b + Math.sqrt(discriminant)) / 2 / a;
        return res;
    }

    public double getRoot2() {
        double discriminant = getDiscriminant();
        if (discriminant < 0) throw new IllegalArgumentException("Discriminant must not be negative");

        double res = (-b - Math.sqrt(discriminant)) / 2 / a;
        return res;
    }
}
```

Check

What will be the value of result after executing the following code?

```
int result = 1;
int i = 1;
while (i <= 4) {
   if (result % 2 == 0) {
      result += i;
   } else {
      result *= i;
   }
   i++;
}
System.out.println(result);
```

Select one:

- (●) a. 20
- ( ) b. 9
- ( ) c. 25
- ( ) d. 15

Clear my choice

Fill in the blanks to complete the following program, which simulates a car rental service by defining a Car class with rental details and methods to update its availability.

```
public class Main {
  public static void main(String[] args) {
    Car rentalCar = new Car("Hongqi", 2023);
    rentalCar.displayDetails();

    // Rent the car and update its status
    rentalCar.setAvailable(false);
    System.out.println("Updated rental status.");
    rentalCar.displayDetails();
  }
}

class Car {
  private String brand;
  private int year;
  private boolean isAvailable = true;

  public Car(__ (1) __ brand, int year) {
    this.brand = brand;
    this.year = year;
  }

  public void displayDetails() {
    System.out.println("Car: " + brand + " (Year: " __ (2) __ ")");
    if (___(3)___) {
      System.out.println("Available: No");
    }
    else {
      System.out.println("Available: Yes");
    }
  }

  public void setAvailable(___(4)___ available) {
    this.isAvailable = available;
  }
```

(1) String

(2) + year +

(3) !isAvailable

(4) boolean

Consider the following code:

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
   if (i % 2 == 0) {
      sum += i * 2;
   } else {
      sum -= i;
   }
}
System.out.println(sum);
```

What will be the output after executing the code?

Select one:

- a. 8
- b. 5
- c. 3
- d. -3

Clear my choice

What will be the range of possible values for num after executing the following code?

```
int num = (int) (Math.random() * 6) + 5;
System.out.println(num);
```

Select one:

- a. 0 to 5
- b. 5 to 11
- c. 5 to 10
- d. 6 to 11

Clear my choice

What will be the output of the following code?

```
public class Main {
   public static void main(String[] args) {
      String str = "JavaProgramming";
      str = str.substring(4).replace('a', 'X');
      str = str.substring(0, str.indexOf('X') + 2);
      System.out.println(str);
   }
}
```

Select one:

- (selected) a. ProgrXm
- b. ProgrXmm
- c. JXv
- d. JXvX

Clear my choice

Which one of the following is the correct way to initialize an array in Java?

Select one:

- a. `double[][] arr = new double[3, 3];`
- b. `int[][] arr = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};`
- c. `String arr[] = new String {"a", "b", "c"};`
- d. `int arr[] = new {1, 2, 3, 4, 5};`

Clear my choice

Which one of the following is valid if statements in Java?

Select one:

- a. `if (y != 0) { System.out.println("y is not zero"); }`
- b. `if (flag == true) { System.out.println("Error detected") };`
- c. `if (age < 18) System.out.println("Underage") else System.out.println("Adult");`
- d. `if (a = 5) { System.out.println("a is 5") };`

Clear my choice

What is the output of the following function?

```
private static int compute(int start, int end, int a[], int b[]) {
  int sum = 0;
  for (int i = start; i <= end; i++) {
    sum += b[i] - a[i];
  }
  return sum;
}

public static void main(String[] args) {
  int[] c = { 2, 3, 5, 7, 93, 4 };
  int[] d = { 23, 4, 9, 3, 40, 33, 1 };

  System.out.println(compute(2, 4, d, c));
}
```

Select one:

- a. 0
- b. 53
- c. -22
- d. -21

Clear my choice

Fill in the blanks to complete the following program, which reads a string, an integer, and a double from user input.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(___(1)___);

      System.out.println("Enter a string:");
      ____(2)___ text = scanner.nextLine();

      System.out.println("Enter an integer:");
      int number = Integer.___(3)___(scanner.nextLine());

      System.out.println("Enter a decimal number:");
      double decimal = Double.____(4)____(scanner.nextLine());
   }
}
```

(1) System.in

(2) String

(3) parseInt

(4) parseDouble

In GDPR, which rights ensures that a data subject can transfer their data to another organization or the data controller?

Select one:

- a. Right of migration
- b. Right to data transfer
- c. Right to data portability
- d. Right of relocation

Clear my choice

What is the most important ethical consideration when handling user data in a software application?

Select one:

- a. Sharing user data with third parties without disclosure.
- b. Collecting as much user data as possible for future use.
- c. Ensuring user data privacy, security, and obtaining informed consent.
- d. Minimizing data encryption to improve performance.

Clear my choice

Design a class named QuadraticEquation for a quadratic equation $ax^2 + bx + c = 0$.
The class contains:

a) Private data fields a, b, and c that represents three coefficients, which can have decimal values.
b) A constructor for the arguments a, b, and c.
c) Three get methods for a, b, and c.
d) A method named getDiscriminant() that returns the discriminant, which is $b^2 - 4ac$.
e) The methods named getRoot1() and getRoot2() for returning two roots of the equation

$$r_1 = \frac{-b+\sqrt{b^2-4ac}}{2a} \text{ and } r_2 = \frac{-b-\sqrt{b^2-4ac}}{2a}$$

These methods are useful only if the discriminant is nonnegative, or a IllegalArgumentException should be thrown with
a message "Discriminant must not be negative".

Given Test cases:
(new QuadraticEquation(1.0, 1.0, -6.0)).getDiscriminant() -> 25.0
(new QuadraticEquation(1.0, 1.0, -6.0)).getRoot1() -> 2.0
(new QuadraticEquation(1.0, 1.0, -6.0)).getRoot2() -> -3.0

For example:

Test	Result
System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getDiscriminant());	25.0
System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getRoot1());	2.0
System.out.println((new QuadraticEquation(1.0, 1.0, -6.0)).getRoot2());	-3.0

Answer: (penalty regime: 0 %)

Reset answer

```java
public class QuadraticEquation {

    private double a;
    private double b;
    private double c;

    public QuadraticEquation(double a, double b, double c) {
        this.a = a;
        this.b = b;
        this.c = c;
    }

    public double getA() {
        return a;
    }

    public double getB() {
        return b;
    }

    public double getC() {
        return c;
    }

    public double getDiscriminant() {
        return b * b - 4 * a * c;
    }

    public double getRoot1() {
        double discriminant = getDiscriminant();
        if (discriminant < 0) throw new IllegalArgumentException("Discriminant must not be negative");

        double res = (-b + Math.sqrt(discriminant)) / 2 / a;
        return res;
    }

    public double getRoot2() {
        double discriminant = getDiscriminant();
        if (discriminant < 0) throw new IllegalArgumentException("Discriminant must not be negative");

        double res = (-b - Math.sqrt(discriminant)) / 2 / a;
        return res;
    }
}
```

Check

A software company discovers that one of their applications has a serious security vulnerability that could expose user data. Fixing the issue would require downtime, which might inconvenience users, and the company is under pressure from stakeholders to meet a tight deadline for a new feature release. What is the most ethical course of action for the company?

Select one:

- (●) a. Delay the feature release and prioritize fixing the security vulnerability, informing users about the issue.
- ( ) b. Keep the vulnerability secret and wait to see if users discover the issue.
- ( ) c. Ignore the security issue as there have been no reported breaches yet.
- ( ) d. Release the new feature on time and fix the security issue in the next update.

Clear my choice